

Distributed Task Allocation Under Temporal Logic and Communication Constraints with Resilient Task Coordination

Abstract—Coordinating task allocation for heterogeneous multi-robot systems under communication and temporal-logic constraints is challenging because plans that are feasible offline may become blocked or unrealizable during execution when robots cannot maintain complete global information. To address this, a distributed task allocation framework with resilient task coordination is developed. The global task is translated into a pruned Büchi automaton, decomposed into executable subtasks, and initially allocated through a consensus-based timetable algorithm. To sustain task coordination under limited communication range, the proposed framework incorporates two complementary mechanisms: a patrol mechanism that actively gathers information to release temporally blocked subtasks, and a resilient task coordination mechanism for collaborative subtasks that integrates leader-based coordination, timeout-triggered recruitment, and free-market-based reassignment. In this way, the framework couples distributed task allocation with resilient task coordination, enabling the robot team to maintain task progress despite incomplete and delayed information. Simulations on 25 benchmark instances and a physical experiment demonstrate the effectiveness of the proposed framework in sustaining task execution under limited communication.

Index Terms—Linear Temporal Logic, Multi-robot Systems, Task Allocation, Multi-robot Coordination.

I. INTRODUCTION

Multi-robot systems (MRS) are increasingly important in applications such as patrol, surveillance, and logistics [1], [2], because robot teams can provide higher efficiency, robustness, and flexibility than a single robot can. Multi-robot task allocation (MRTA) is therefore a fundamental problem in MRS. Existing task-allocation methods are generally divided into centralized and distributed approaches. Centralized methods compute assignments through a coordinator with global information [3], [4], but they often suffer from high computational cost and single-point-of-failure issues, which limit scalability and robustness in dynamic environments. By contrast, distributed methods allow robots to make decisions based on local information and limited communication, including auction-based algorithms [5], [6], contract-net protocols [7], and rule-based methods [8]. These methods are especially attractive in communication-constrained scenarios such as security patrol and post-disaster search and rescue [9].

However, many practical multi-robot missions are not only communication-constrained, but also subject to temporal requirements such as ordering, persistence, and synchronization [10], which makes MRTA more challenging. To specify such requirements formally and precisely, recent studies have increasingly adopted formal languages [11], among which linear temporal logic (LTL) is one of the most widely used [12]. LTL provides a rigorous and unambiguous framework for task specification. For example, Li et al. [13] studied heterogeneous MRTA under global LTL specifications with variable robot capabilities and inter-task constraints. Yu et al. [14] investigated

fully distributed LTL-based multi-robot motion coordination using only local information, and employed online conflict detection and local replanning to preserve safety and specification feasibility. Despite this progress, sustaining global LTL missions in a distributed manner remains challenging under limited communication, because robots cannot maintain complete and timely team-state information during execution. Motivated by this challenge, this work develops a distributed task allocation framework with resilient task coordination under temporal logic and communication constraints.

Under LTL constraints, existing task allocation and planning methods can generally be classified into top-down and bottom-up approaches. In top-down methods, a global LTL formula is used to specify the overall mission, and a centralized planner decomposes it into robot-level plans. A common strategy is to construct a product automaton from the nondeterministic Büchi automaton (NBA) derived from the global LTL formula and the discrete transition systems of the robots, and then solve the resulting graph-search problem [15], [16]. However, this approach suffers from severe state explosion in multi-robot settings and is difficult to adapt to dynamic execution scenarios involving robot addition or removal. Sampling-based methods [17], [18] improve upon the above methods by constructing generation trees from NBA samples rather than from the full product automaton. However, such methods typically require the candidate robots for each subtask to be specified in advance. Luo et al. [19] formulated temporal-logic task allocation as a mixed-integer programming problem; however, it is difficult to adapt online to practical execution contingencies such as robot failures. Chen et al. [20] proposed an incremental decision-tree-based method that supports online adaptation, but it relies on global information. Recognizing the parallel and concurrent nature of MRS, some studies decompose a global specification into multiple subtasks using automaton transition relations to increase parallelism. Schillinger et al. [21], [22] decomposed the global task into a set of parallel subtasks. However, this method does not explicitly handle collaborative subtasks. Based on these works, Liu et al. [23], [24] further improved efficiency by representing subtask temporal relations with posets. Recent studies also combine large language models with optimization-based schedulers, such as DEXTER-LLM [25]. Overall, although advanced top-down methods are effective at deriving task assignments from global temporal-logic specifications, can handle collaborative subtasks, and in some cases allow online adaptation, they still largely rely on centralized coordination and reliable global information. As a result, they are difficult to deploy with limited communication.

Bottom-up approaches, in contrast, usually assume that each robot is assigned a local temporal-logic task, and distributed coordination is then achieved through neighbor communica-

tion and online replanning. Filippidis et al. [26] proposed a distributed planning framework under temporal logic constraints that enables robots to exchange information and verify conflicts online. Each robot can communicate only with neighbors within its communication range, and a dedicated “follower” robot is used to establish and maintain communication links when interaction is required. In [27], [28], each robot updates its knowledge based on local observations and propagates it through neighbor communication; with newly received knowledge, robots confirm and adjust their action paths online. However, such neighbor-only communication cannot guarantee that all robots required for a collaborative subtask can mutually communicate when needed. For scenarios with collaborative tasks, Tian et al. [29] proposed a distributed cooperative computation framework that decomposes task constraints via an interaction-edge product automaton, enabling robots to synthesize local trajectories in parallel using only neighborhood interaction information. Most existing work focuses on avoiding physical collisions while overlooking task conflicts. Chen et al. [30] addressed task-level conflict resolution under communication constraints by predicting the goals of robots beyond communication range.

Compared with top-down approaches, these methods are generally more robust to communication constraints. However, many of them rely on a key assumption that local tasks are known or preassigned in advance. This assumption limits their applicability in scenarios where only a global mission is available. Moreover, when communication is restricted to local neighborhoods, information exchange alone may be insufficient to reliably support collaborative subtasks that require multiple robots. Therefore, existing methods still leave an important gap for scenarios in which robots must derive executable local behavior from a global specification and sustain task execution under limited communication range.

This gap becomes particularly significant in practical applications such as industrial inspection, photovoltaic power station patrol, and large-scale facility monitoring, where communication is restricted and mission requirements are more naturally specified as a global LTL formula rather than as preassigned local tasks. In such settings, the main challenge lies not only in obtaining a feasible initial task assignment, but also in sustaining task coordination during execution under limited communication range. Because robots cannot maintain a complete and timely view of the team state, a subtask that is feasible at the planning stage may later become unrealizable because of sudden robot failures or other unexpected events. In particular, two major challenges arise during mission execution. First, a robot may be unable to verify whether the temporal prerequisites of one of its pending subtasks have been satisfied, which leads to task blocking. Second, a collaborative subtask may fail to start because the required robots do not arrive in time. Existing recovery strategies, including role reassignment, task reprioritization, and subtask reissuing [31]–[33], can alleviate these issues to some extent, but they often operate from fragmented local views. Consequently, repeated failures due to the lack of global information or delayed information propagation may substantially increase the overall mission completion time.

Motivated by these challenges, this work develops a distributed task allocation framework with resilient task coordination under temporal logic and communication constraints. In particular, the proposed method explicitly addresses two issues that are not systematically handled in prior work: task blocking caused by unconfirmed temporal prerequisites and collaborative-subtask failures caused by incomplete team-state information. To address task blocking caused by unconfirmed prerequisite completion, a patrol mechanism is introduced to actively gather and relay task-state information. To improve the resilience of task coordination, especially for collaborative subtasks, a resilient task coordination mechanism is developed by integrating leader-based coordination, timeout-triggered recruitment, and free-market-based reassignment. Here, the free market is a designated rendezvous region where robots synchronize state information and recover failed subtasks based on a more complete view of the team state. By explicitly incorporating these coordination and recovery mechanisms into execution, the proposed framework improves the robustness of global mission execution under limited communication. The main contributions of this work are summarized as follows:

- We propose a distributed task allocation framework that integrates task allocation and resilient task coordination under temporal logic and communication constraints, enabling robot teams to sustain task execution from a global specification using local information.
- To address task blocking under limited communication, a patrol mechanism is proposed to actively gather information when prerequisite subtasks are overdue but completion is unconfirmed.
- To strengthen resilient task coordination for collaborative subtasks, two complementary mechanisms are designed: a leader-based coordination with timeout-triggered recruitment to reduce subtask failures, and a free-market-based reassignment to mitigate significant delays when failures occur.
- Theoretical analysis establishes completeness and task-completion guarantees under the assumptions of this paper.

Notations: Let $\mathbb{N}$ and $[N]$ denote the set of natural numbers and the shorthand notation for $\{1, \dots, N\}$, respectively. Given a set $\mathcal{A}$, let $|\mathcal{A}|$ and $2^{\mathcal{A}}$ denote the cardinality and power set of $\mathcal{A}$, respectively.

II. PRELIMINARIES

LTL formulas are constructed from atomic propositions $\mathcal{AP}$ using Boolean operators and temporal operators. Atomic propositions represent basic Boolean variables whose values are evaluated as either true or false. Detailed content can be found in [12]. LTL formulas can be translated into NBA.

Definition 1 (Nondeterministic Büchi Automaton). *A non-deterministic Büchi automaton (NBA) is a tuple $\mathcal{B} = (\mathcal{S}, \mathcal{S}_0, \Sigma, \delta, \mathcal{F})$, where $\mathcal{S}$ is a finite set of states, $\mathcal{S}_0 \subseteq \mathcal{S}$ is the set of initial states, $\Sigma = 2^{\mathcal{AP}}$ is the input alphabet, $\delta : \mathcal{S} \times \Sigma \rightarrow 2^{\mathcal{S}}$ is the transition function, and $\mathcal{F} \subseteq \mathcal{S}$ is the set of accepting states.*

Given an infinite word $\omega = \sigma_0\sigma_1\sigma_2\ldots$ with $\sigma_i \in \Sigma$, a run $\rho = s_0s_1s_2\ldots$ of the NBA $\mathcal{B}$ over ω is an infinite state sequence such that $s_0 \in S_0$ and $s_{i+1} \in \delta(s_i, \sigma_i)$ for all $i \geq 0$. The run ρ is accepting if it visits states in F infinitely often. Accepting runs can be represented in a prefix-suffix structure, where the prefix w_{pre} runs from an initial state to an accepting state, and the suffix w_{suf} forms a cycle containing at least one accepting state.

This paper focuses on a special class of LTL formulas called syntactically co-safe LTL (sc-LTL) [34]. sc-LTL formulas restrict temporal operators to $\bigcirc$ (next), $\cup$ (until), and $\diamond$ (eventually), and must be written in a positive normal form.

III. PROBLEM STATEMENT

A. Environment and Multi-Robot System

In a discrete workspace $\mathcal{W} \subseteq \mathbb{R}^2$, let $\mathcal{L} = \{l_1, l_2, \ldots, l_M\}$ be M pairwise disjoint target regions, i.e., $l_i \cap l_j = \emptyset$ for all $i \neq j$, where $i, j \in [M]$. Let $\mathcal{U}$ denote uninteresting regions, such as forbidden areas, and assume $l_i \cap \mathcal{U} = \emptyset$.

Given workspace $\mathcal{W}$, we consider a heterogeneous robot team $\mathcal{R} = \{r_1, r_2, \ldots, r_N\}$, where N is the number of robots. The robots have n_t different types, where each robot belongs to exactly one type. Let $\mathcal{T} = [n_t]$ denote the type index set. For each $i \in \mathcal{T}$, let $\mathcal{R}_i \subseteq \mathcal{R}$ denote the set of robots of type i . Then $\mathcal{R} = \bigcup_{i=1}^{n_t} \mathcal{R}_i$ and $\mathcal{R}_i \cap \mathcal{R}_j = \emptyset, \forall i \neq j$. For any robot r_m , its velocity and position are denoted by v_m and x_m , respectively. Consider a time sequence $\{\tau_n = n\Delta t\}_{n \in \mathbb{N}}$ with sampling interval Δt . For convenience, for any quantity y , $y(\tau_n)$ denotes the value or state of y at time τ_n . In other words, the argument τ_n indicates the realization of the corresponding variable evaluated at τ_n .

Assuming communication range $D \in \mathbb{R}^+$, robots can only communicate with neighbors within this range. The neighbor set of robot r_m at time τ_n is defined as: $\mathcal{N}_m(\tau_n) = \{r_n \mid \|x_m(\tau_n) - x_n(\tau_n)\| \leq D\}$. The communication frequency is denoted by f_{com} in Hz.

B. Problem Description

To better describe the problem, we first introduce the atomic propositions. Let $a_{i,w}^m$ represent the action required for task completion, indicating that w robots of type i must execute this action at region $l_m \in \mathcal{L}$, where $i \in \mathcal{T}$, $w \in \mathbb{N}$, and $w \leq |\mathcal{R}_i|$. The following atomic propositions are then introduced: i) the atomic proposition p_m is true when any robot is located in region l_m , and false otherwise; ii) the atomic proposition $l_{i,w}^m$ is true iff action $a_{i,w}^m$ has been completed.

To facilitate task allocation under temporal logic constraints, a task model is introduced to explicitly capture the relationships among subtasks, actions, and workspace information. This abstraction provides a unified representation for subsequent task allocation.

Definition 2 (Task Model). *The task model is defined as $TM = (\mathcal{Q}, \mathcal{A}, \mathcal{W}, \text{LA}, \text{LC}, \text{LW})$, where $\mathcal{Q} := \{q_i\} \cup \{\varepsilon_q\}$ is the set of subtasks and ε_q is the null subtask; $\mathcal{A} := \{a_{i,w}^m \mid m \in [M], i \in [n_t], w \in [|\mathcal{R}_i|]\} \cup \{\varepsilon_a\}$, where ε_a is the empty action. For notational simplicity, an action in $\mathcal{A}$ can*

be written as a_j , where j is the index. $\mathcal{W}$ is the workspace; $\text{LA} : \mathcal{Q} \rightarrow 2^{\mathcal{A}}$ associates each subtask with a set of actions, i.e., $\text{LA}(q_i) \subseteq \mathcal{A}$; $\text{LC} : \mathcal{A} \rightarrow \mathbb{R}_{\geq 0}$ indicates the cost required to execute an action; and $\text{LW} : \mathcal{A} \rightarrow \mathcal{W}$ specifies the target region for an action (excluding ε_a).

Given workspace $\mathcal{W}$ and NBA $\mathcal{B}$ corresponding to the LTL-specified global task φ , the plan for robot r_k is defined as a subtask-action-time sequence $\Pi_k = \Pi_{0,k}\Pi_{1,k}\cdots\Pi_{h_k,k}$, where each entry $\Pi_{i,k}$ is a subtask-action-time tuple associated with robot r_k , and h_k denotes the length of the plan. More specifically, $\Pi_{i,k} = (q_i, a_i, t_{i,k})$, where $q_i \in \mathcal{Q}$, $a_i \in \text{LA}(q_i)$, and $t_{i,k}$ denote the assigned subtask, the corresponding action, and the action start time of the i -th entry in the plan of robot r_k , respectively. In particular, the action of $\Pi_{0,k}$ is ε_a and the subtask is ε_q . To simplify notation, we use $\Pi_{i,k}(q)$ to denote the subtask component of $\Pi_{i,k}$ and $\Pi_{i,k}(a)$ to denote its action component in the remainder of this paper.

Example 1. *Consider a heterogeneous robot group $\mathcal{R} = \{r_1, r_2, r_3, r_4\}$, including two type-1 (e.g., inspection) and two type-2 (e.g., maintenance and manipulation) robots, performing distributed tasks in a large-scale industrial park. The target regions are $\mathcal{L} = \{l_1, l_2, l_3, l_4\}$. The task requires two type-1 robots to go to l_1 for a joint inspection of an abnormal facility status, and one type-2 robot must travel to a local service station at l_2 to perform an on-site handling operation. In addition, one type-1 and one type-2 robot must go to l_3 for a coordinated repair task. During this coordinated-repair period, no robot may pass through region l_4 , which represents a hazardous area. Let p_4 denote the atomic proposition that any robot enters region l_4 . The task can be described by the following sc-LTL formula.*

$$\varphi = \diamond l_{1,2}^1 \wedge \diamond l_{2,1}^2 \wedge \diamond (l_{1,1}^3 \wedge l_{2,1}^3 \wedge \neg p_4) \quad (1)$$

Based on the above, the problem studied in this work is described as follows.

Problem 1. *Given a global sc-LTL task specification φ , workspace $\mathcal{W}$, and a team of heterogeneous robots $\mathcal{R}$, the goal is to develop a distributed task allocation and coordination strategy under limited communication range. Specifically, the strategy should generate a plan Π_k for each robot $r_k \in \mathcal{R}$ and sustain task coordination during execution despite incomplete and delayed information. Let $\Pi := \{\Pi_k\}_{r_k \in \mathcal{R}}$ denote the collection of local plans. The objective is that Π satisfies task φ . The following assumptions are made.*

- i) *a robot can determine whether the task at a target region has been completed upon arrival;*
- ii) *any robot within the broadcast range of a sender (another robot) will instantly receive its messages;*
- iii) *the robot team contains sufficient robots.*

IV. FRAMEWORK OVERVIEW

As shown in Fig. 1, the proposed framework consists of two parts: task preprocessing and distributed allocation with resilient task coordination.

In the task preprocessing part, the global sc-LTL specification is first translated into an NBA. The automaton is then

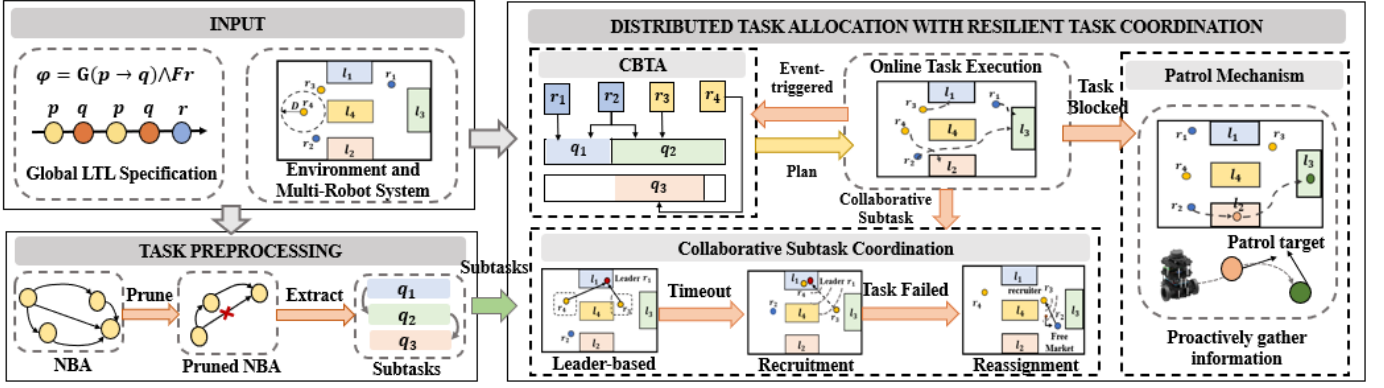


Fig. 1. Framework of the method. The framework consists of two stages: task preprocessing and distributed task allocation with resilient task coordination.

pruned by removing infeasible and decomposable transitions, and the resulting pruned automaton is further decomposed into executable subtasks together with their temporal dependencies.

In the second part, robots first generate initial plans through the consensus-based timetable algorithm (CBTA), which allocates optional subtasks. Since communication is limited, each robot maintains and broadcasts not only its own task state but also the peer information it has received. Based on the locally information, robots maintain their local views of subtask progress and update their ordinary local plans through an event-triggered replanning mechanism. This mechanism is used only for online adaption outside the free market.

For collaborative subtasks, the first robot arriving at the target region acts as a temporary leader to coordinate team formation and synchronize task execution. If all required robots arrive within the collaboration waiting time, the leader broadcasts a start signal and the collaborative subtask proceeds as planned. Otherwise, timeout-triggered recruitment is activated. The leader first attempts local recovery by recruiting available and type-compatible robots that are reachable through direct or multi-hop communication. If local recovery is insufficient, a recruiter may be dispatched to the free market. If the required team still cannot be formed within the prescribed coordination horizon, the leader declares the collaborative subtask failed, and the failed subtask will be handled through free-market-based reassignment.

In addition, when a robot cannot confirm that the temporal prerequisites of its pending subtasks have been satisfied, a patrol mechanism is activated to actively gather the missing task-state information.

Overall, the framework starts from a global temporal logic task, derives distributed executable subtasks, generates initial plans, and then sustains task progress online through event-triggered replanning, collaborative coordination, and patrol-based information gathering under limited communication.

V. TASK PREPROCESSING

To address Problem 1, we preprocess φ . As shown in Fig. 1, preprocessing has two phases: i) NBA pruning; and ii) task decomposition on the pruned NBA to obtain the subtask set $\mathcal{Q}$ and the temporal relations among subtasks.

A. Pruning of NBA

The LTL formula is first converted into an NBA $\mathcal{B}$ and then pruned to remove redundancies, yielding a simplified automaton $\mathcal{B}^-$. Specifically, there are two types of pruning:

- *Removal of infeasible transitions*: removing transitions that cannot be realized, e.g., when the required number or types of robots exceed those available.
- *Elimination of decomposable transitions*: Decomposable transitions are removed to improve task decomposition effectiveness [35].

A decomposable transition is defined as follows.

Definition 3 (Decomposable Transition). *In NBA $\mathcal{B}$, a transition from state s_i to s_j is called decomposable if there exists another state s_k such that, for any $\sigma_{ik}, \sigma_{kj} \subseteq \Sigma$ satisfying $s_k \in \delta(s_i, \sigma_{ik})$ and $s_j \in \delta(s_k, \sigma_{kj})$, it holds that $s_j \in \delta(s_i, \sigma_{ik} \cup \sigma_{kj})$.*

Lemma 1: For any feasible word w accepted by $\mathcal{B}$, there exists a corresponding word w' accepted by $\mathcal{B}^-$.

Proof: Consider an accepted word $w = \dots \sigma_n \dots$ with accepting run $\rho = \dots s_i s_j \dots$ in $\mathcal{B}$, where $s_j \in \delta(s_i, \sigma_n)$. The first pruning step removes only infeasible transitions, so every feasible accepted word is preserved. For the second pruning step, consider two cases. For Case 1, if ρ contains no decomposable transitions, then all transitions of ρ remain in $\mathcal{B}^-$; hence ρ is also an accepting run in $\mathcal{B}^-$ and we can set $w' = w$. For Case 2, if a transition from s_i to s_j is removed as decomposable, there exists a state s_k and labels σ_{ik}, σ_{kj} such that $s_k \in \delta(s_i, \sigma_{ik})$, $s_j \in \delta(s_k, \sigma_{kj})$, and $\sigma_{ik} \cup \sigma_{kj} = \sigma_n$. Therefore, the transition $s_i \rightarrow s_j$ can be replaced by two transitions, $s_i \rightarrow s_k$ and $s_k \rightarrow s_j$. If both transitions $s_i \rightarrow s_k$ and $s_k \rightarrow s_j$ are non-decomposable, this reduces to Case 1. Otherwise, the same decomposition argument is applied recursively. Because a transition cannot be decomposed indefinitely into realizable finite expressions, this recursive process terminates. The final constructed run has no decomposable transitions, so it reduces to Case 1 and is accepted by $\mathcal{B}^-$.

B. Task Decomposition

Following [36], $\mathcal{B}^-$ is decomposed into subtasks and a relaxed partial order is extracted. A depth-first search on

$\mathcal{B}^-$ generates an accepting path and its corresponding task sequence. Starting from this ordered sequence, subtasks are iteratively swapped and each new path is checked for acceptability. This process relaxes unnecessary temporal constraints and yields a logically consistent subtask set that supports parallel execution, together with a relaxed partial order.

Example 2. For the task in Example 1, the subtask set corresponding to φ is given by $\Omega_\varphi = \{q_1, q_2, q_3\}$, where $q_1 = \{1, \{l_{1,2}^1\}, \{\}\}$, $q_2 = \{2, \{l_{2,1}^2\}, \{\}\}$, $q_3 = \{3, \{l_{1,1}^3, l_{2,1}^3\}, \{p_4\}\}$. Each subtask is represented by three entries: the first entry (e.g., 1, 2, 3) is the subtask index, the second set contains the actions to be executed, and the third set contains atomic propositions required on the self-loop.

VI. DISTRIBUTED ALLOCATION WITH RESILIENT TASK COORDINATION

This section describes how feasible plans are generated under limited communication, and how these plans are executed under limited communication range.

A. Consensus-based Timetable Algorithm

To prevent robots from violating temporal logic constraints, the optional subtask set $\mathcal{O}_k(\tau_n)$ for robot r_k is defined.

Definition 4 (Optional Subtask Set). At time τ_n , the optional subtask set $\mathcal{O}_k(\tau_n)$ for robot r_k includes all subtasks $q_i \in \mathcal{Q}$ that satisfy: i) the numbers or types of robots committed to q_i have not yet met the task requirements; ii) r_k is unassigned and capable of q_i ; iii) temporal constraints are satisfied.

Note that robots committed to q_i include those traveling to, waiting in, or executing the subtask at the target region. To reduce failures in collaborative subtasks, a robot commits only when its currently available information indicates that enough planned participants, in both number and required types, are already allocated to that subtask.

Allocation within $\mathcal{O}_k(\tau_n)$ is performed using CBTA [5], which can efficiently allocate tasks by iterative timetable construction and consensus-reaching among robots. The cost of the current plan for robot r_k , i.e., the plan completion time, is defined below.

$$\text{Cost}(\Pi_k(\tau_n)) = t_{h_k(\tau_n),k} - t_{0,k} + \text{LC}(\Pi_{h_k(\tau_n),k}(a)) \quad (2)$$

Here, $h_k(\tau_n)$ is the current plan length of robot r_k at time τ_n ; $t_{h_k(\tau_n),k}$ is the start time of the last action in $\Pi_k(\tau_n)$; $t_{0,k}$ is the start time of the plan; and $\text{LC}(\Pi_{h_k(\tau_n),k}(a))$ is the execution cost of the last action. For each $q_i \in \mathcal{O}_k(\tau_n)$ and action $a_j \in \text{LA}(q_i)$, candidate insertion positions after $\Pi_{0,k}(\tau_n)$ are evaluated by minimizing marginal cost, where $\Pi_{0,k}(\tau_n)$ is the initial tuple of the plan for robot r_k at time τ_n . The time component $t_{i,k} \in \Pi_{i,k}(\tau_n)$ is defined below.

$$t_{i,k} = \begin{cases} t_{0,k} + t_{\text{left},k} + \frac{d_{i,k}}{v_k}, & \text{if } i = 1 \\ t_{i-1,k} + \text{LC}(\Pi_{i-1,k}(a)) + \frac{d_{i-1,i}}{v_k}, & \text{otherwise} \end{cases} \quad (3)$$

where $t_{\text{left},k}$ is the remaining time for the current subtask; $d_{i,k}$ is the distance from $x_k(\tau_n)$ to the target region

of $\Pi_{i-1,k}(q)$; and $d_{i-1,i}$ is the distance between the target regions of $\Pi_{i-1,k}(q)$ and $\Pi_{i,k}(q)$. To enforce robot-quantity requirements, Eq. (2) is augmented with a penalty $M \sum_{i \in [h_k(\tau_n)]} n_{\text{req}}(\Pi_{i,k}(q))$, where $n_{\text{req}}(\Pi_{i,k}(q))$ is the number of robots required to accomplish subtask $\Pi_{i,k}(q)$ and M is a large constant.

Assume that $\mathcal{P}_k(\tau_n)$ is the set of plans received by robot r_k at time τ_n . Task-allocation conflicts are then resolved based on these shared plans as follows. For each candidate subtask $q_i \in \mathcal{O}_k(\tau_n)$ and each action $a_j \in \text{LA}(q_i)$, robot r_k first counts the number of robots currently assigned to execute a_j according to $\mathcal{P}_k(\tau_n)$. Let $n_{\text{req}}^k(a_j)$ denote the number of robots that r_k estimates to be required for a_j . If the number of assigned robots exceeds $n_{\text{req}}^k(a_j)$, an over-assignment conflict is detected. In this case, only the first $n_{\text{req}}^k(a_j)$ robots with the earliest expected action start times are retained. Otherwise, no task-allocation conflict is identified, and all currently assigned robots are retained. See [5] for algorithmic details.

Example 3. For robot r_1 in Example 1, suppose its optional subtask set is $\mathcal{O}_1(\tau_n) = \{q_1, q_3\}$ and the initial plan is $\Pi_1(\tau_n) = (\varepsilon_q, \varepsilon_a, \tau_n)$, where ε_q and ε_a denote empty subtask and action, respectively. The actions of robot r_1 for q_1 and q_3 are $a_{1,2}^1$ and $a_{1,1}^3$ with $\text{LC}(a_{1,2}^1) = 2.6$ and $\text{LC}(a_{1,1}^3) = 2.5$, where 2.6 and 2.5 denote the time required to complete the corresponding actions. First, $a_{1,2}^1$ is inserted; at this stage it can only be placed after $\Pi_{0,1}$, yielding $\Pi_1(\tau_n) = (\varepsilon_q, \varepsilon_a, \tau_n)(q_1, a_{1,2}^1, \tau_n + 5.2)$. After that, $a_{1,1}^3$ is inserted; at this stage, it can be inserted either between $\Pi_{0,1}$ and $\Pi_{1,1}$ or after $\Pi_{1,1}$, and the corresponding marginal costs are 16.7 and 7.9, respectively. Then, $\Pi_1(\tau_n)$ is further updated to $(\varepsilon_q, \varepsilon_a, \tau_n)(q_1, a_{1,2}^1, \tau_n + 5.2)(q_3, a_{1,1}^3, \tau_n + 13.2)$.

B. Event-Triggered Replanning Mechanism

The replanning mechanism described in this subsection is the CBTA-based update of local plans during online execution. It is used to revise ordinary local task assignments when newly received information changes a robot's view of the current task state. This mechanism is distinct from the free-market reassignment process for failed collaborative subtasks. In particular, CBTA replanning is used for online adaptation outside the free market, whereas once a robot enters the free market, failed collaborative subtasks are handled by the free market-based reassignment mechanism.

During execution, each robot r_k maintains and periodically broadcasts a robot-state information set $\mathcal{S}_k(\tau_n)$, which records the information collected by r_k at time τ_n . In particular, each robot broadcasts not only its own state but also the peer states it has received, thereby enabling multi-hop information propagation. The robot-state information set is defined as $\mathcal{S}_k(\tau_n) = \{\mathcal{S}_{m|k}(\tau_n) \mid r_m \in \mathcal{R}, m \neq k\}$. Each element $\mathcal{S}_{m|k}(\tau_n)$ is defined as follows: $\mathcal{S}_{m|k}(\tau_n) = \{\tau_m, q_m, a_m, t_a^m, t_s^m, x_m(\tau_n), \Pi_m(\tau_n)\}$. Here, $\mathcal{S}_{m|k}(\tau_n)$ records the latest status of robot r_m as received by r_k at time τ_n . Specifically, τ_m is the generation timestamp of the latest status message from r_m ; q_m is the subtask currently committed by r_m ; $a_m \in \text{LA}(q_m)$ is the concrete action role undertaken by r_m for q_m ; t_a^m is the estimated remaining travel

time from the current estimated position of r_m to the target region of q_m ; and t_s^m is the estimated task start time. Let $\mathcal{A}_m$ denote the set of estimated arrival times of other robots assigned to the same task region. Then t_s^m is defined as the difference between the estimated arrival time of r_m itself and the $n_{\text{req}}(a_m)$ -th smallest element in $\mathcal{A}_m$, and is always non-negative. In addition, $x_m(\tau_n)$ denotes the position of r_m at time τ_n , and $\Pi_m(\tau_n)$ denotes the plan of r_m at time τ_n .

To reduce unnecessary computation under limited communication, CBTA replanning is triggered only by events that update a robot's local task view or indicate changes in task-execution status. Specifically, these events include the reception of task-state information with newer timestamps, completion or failure of the robot's current subtask, task-allocation conflicts, and failure inference for non-collaborative subtasks, as illustrated in Fig.2.

1) *Conflict-Triggered Replanning*: During execution, limited communication may lead to task conflicts. To address this, a conflict-triggered replanning mechanism is introduced.

Each robot classifies subtasks into three categories at time τ_n : i) unassigned subtasks, whose currently committed robots are still insufficient in number or type, so additional robots may still join; ii) completed subtasks, whose required actions have already been executed and therefore no longer require allocation; and iii) confirmed subtasks, whose current commitments have been verified to satisfy all participation requirements and are thus ready for execution or are already under execution. Let $\mathcal{T}_{\text{not}}^k(\tau_n)$ and $\mathcal{T}_{\text{already}}^k(\tau_n)$ denote the unassigned-subtask set and completed-subtask set maintained by robot r_k , respectively. During communication, robots exchange these sets and merge newly received updates. In this way, the optional subtask set is maintained online and remains consistent with the latest distributed task state.

Based on the robot-state information set $\mathcal{S}_k(\tau_n)$, the unassigned-subtask set $\mathcal{T}_{\text{not}}^k(\tau_n)$, and the completed-subtask set $\mathcal{T}_{\text{already}}^k(\tau_n)$, robot r_k periodically checks for conflicts while executing $a_j \in \text{LA}(q_i)$. Two types of conflicts are considered. i) Conflict caused by task completion: At each check, r_k verifies whether q_i has entered $\mathcal{T}_{\text{already}}^k(\tau_n)$. If so, the current commitment is immediately invalidated, since further motion for q_i is redundant and may delay other pending tasks. Robot r_k then stops and replans. ii) Over-assignment: If the number of robots committed to a_j exceeds the required cardinality $n_{\text{req}}(a_j)$, robot r_k ranks all committed robots by their remaining travel time to the target region, with smaller t_a^m taking higher priority, and keeps only the first $n_{\text{req}}(a_j)$ robots as valid executors of a_j . If multiple robots have the same t_a^m , ties are broken by robot ID in ascending order. If r_k is not selected after this filtering step, it immediately stops the current commitment and replans.

2) *Replanning for Non-Collaborative Subtasks*: This replanning is applied only to non-collaborative subtasks, because collaborative subtasks are handled by a dedicated resilient task coordination mechanism. For a non-collaborative subtask, the assigned robot may become unavailable during execution because of faults or other unexpected events and may therefore fail to complete the subtask. Under limited communication, other robots cannot directly distinguish such execution failure

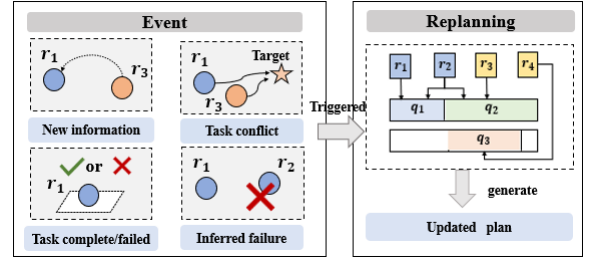


Fig. 2. Schematic illustration of the event-triggered replanning mechanism

from delayed or incomplete information propagation. Therefore, each robot applies a timeout-based rule to assess whether the corresponding task-state record remains valid and, if necessary, triggers CBTA-based replanning.

Even for non-collaborative subtasks, delayed and incomplete information propagation may cause the same subtask to be assigned to multiple robots. Let $\mathcal{H}_k(q_m) = \{h \mid \mathcal{S}_{h|k}(\tau_n) \in \mathcal{S}_k(\tau_n), q_h = q_m\}$ denote the robot-index set induced by the state records in $\mathcal{S}_k(\tau_n)$, where q_h is the subtask component in $\mathcal{S}_{h|k}(\tau_n)$. For each $h \in \mathcal{H}_k(q_m)$, robot r_k computes a timeout candidate $T_{\text{late}}^{k,h}$ by Eq. (5) and uses the largest one as the expiry threshold for q_m . The expiry threshold is defined below.

$$T_{\text{late}}^k(q_m) = \max_{h \in \mathcal{H}_k(q_m)} T_{\text{late}}^{k,h}. \quad (4)$$

The timeout candidate is defined below.

$$T_{\text{late}}^{k,h} = \alpha_{\text{late}} \left(1 + \left\lceil \frac{d_{k,h}}{D} \right\rceil \right) \max \left(T_{\text{enc}}, \frac{1}{f_{\text{com}}} \right) + t_s^h + \max_{a_i \in \text{LA}(q_m)} \text{LC}(a_i) \quad (5)$$

The average encounter time is defined below.

$$T_{\text{enc}} = \frac{1}{4Dv_a\rho_r} \quad (6)$$

where α_{late} is a tuning parameter. In this paper, α_{late} is set to 2.5 based on experimental results. The value is chosen to be greater than 1 so that the practical information expiration time remains above its theoretical estimate. $d_{k,h}$ denotes the distance between robot r_k 's current position and robot r_h 's estimated position. The position of r_h is estimated based on its last known position x_h at the most recent communication time, together with its task plan. The estimation is carried out as follows: the current time is compared with the subtask start time in the plan to determine which task the robot is executing. If the current time exceeds the planned completion time of that task, the robot is considered to be in the free market. Otherwise, the robot is assumed to move at a constant speed along a straight line between task points, and its current position is estimated accordingly. T_{enc} denotes the average encounter time [37], where v_a is the average speed of robots and ρ_r is the robot density. Specifically, if current time τ_{now} satisfies $\tau_{\text{now}} - \tau_m > T_{\text{late}}^k(q_m)$, robot r_k treats the state record of robot r_m for subtask q_m as outdated. If robot r_k is capable of executing the corresponding action, q_m is non-collaborative, and $q_m \neq q_k$, then robot r_k triggers CBTA-based replanning, after which the updated local plan may take over q_m . Here,

τ_m is the timestamp of the stored state record, and $T_{\text{late}}^k(q_m)$ is the expiry threshold adopted by robot r_k for subtask q_m .

Note that the event-triggered CBTA replanning mechanism is applied only outside the free market. Once a robot enters the free market, failed collaborative subtasks are no longer handled through CBTA-based local plan updates, but through the dedicated free-market reassignment mechanism.

C. Collaborative Subtask Coordination

1) *Free Market*: The free market is a designated rendezvous region used for global information synchronization and failed-subtask reassignment. A robot enters this region in one of three situations: it has no executable pending subtask unless all subtasks have already been completed, its assigned optional subtask cannot proceed because of failure or missing collaborators, or it is explicitly dispatched by a leader as an emergency recruiter. Robots broadcast the ID set of robots currently in the free market, denoted by $\mathcal{R}_f$, along with the corresponding timestamp. Upon receiving this information, each robot updates its locally maintained free-market ID set $\mathcal{R}_f$ using the newest timestamp.

The free-market location is computed by entropy-based weighting of subtask information. For each subtask $q_i \in \mathcal{Q}$, two indicators are considered: the required robot count N_i and the number of successor subtasks $N_{\text{suc},i}$. Their normalized forms are denoted by $\nu_{i,1}$ and $\nu_{i,2}$, respectively. They are defined below.

$$\nu_{i,1} = \frac{N_i}{\sum_{q_j \in \mathcal{Q}} N_j}, \quad \nu_{i,2} = \frac{N_{\text{suc},i}}{\sum_{q_j \in \mathcal{Q}} N_{\text{suc},j}}. \quad (7)$$

The entropy weight of indicator k is denoted by w_k , where $k = 1$ corresponds to the normalized required robot count and $k = 2$ corresponds to the normalized successor count. The normalized entropy e_k is defined below.

$$e_k = -\frac{1}{\ln |\mathcal{Q}|} \sum_{q_i \in \mathcal{Q}} \nu_{i,k} \ln \nu_{i,k}, \quad k \in \{1, 2\}, \quad (8)$$

The entropy weight w_k is defined below.

$$w_k = \frac{1 - e_k}{\sum_{m=1}^2 (1 - e_m)}, \quad k \in \{1, 2\}. \quad (9)$$

Here, $|\mathcal{Q}|$ is the number of subtasks, and the denominator ensures that $\sum_{k=1}^2 w_k = 1$. The entropy-weighted importance of subtask q_i is defined below.

$$s_i = w_1 \nu_{i,1} + w_2 \nu_{i,2}, \quad (10)$$

Its normalized score is defined below.

$$\bar{s}_i = \frac{s_i}{\sum_{q_j \in \mathcal{Q}} s_j}. \quad (11)$$

Here, s_i is the unnormalized importance score of subtask q_i , and $\bar{s}_i$ is its normalized score. This normalization ensures that $\sum_{q_i \in \mathcal{Q}} \bar{s}_i = 1$, so $\bar{s}_i$ can be directly used as a weight in center computation. Let x_i denote the geometric center of the region associated with subtask q_i . The center of the free market is defined below.

$$x_{\text{free}} = \sum_{q_i \in \mathcal{Q}} \bar{s}_i x_i. \quad (12)$$

This expression is a weighted average of subtask-region centers. A larger value of $\bar{s}_i$ means that subtask q_i has a stronger influence on x_{free} . If x_{free} lies in a forbidden region or outside the workspace, the final free-market location is set to the nearest feasible point. In this work, a robot is regarded as having arrived at the free market when it reaches a feasible point that is as close as possible to the free-market center and lies within the communication range of at least one robot already in the free market, if such a robot exists.

2) *Leader-based Collaborative Subtask Coordination*: For collaborative subtasks, robots adopt a leader-based coordination strategy. One robot serves as a temporary leader to collect arrival information, synchronize decisions, and issue recruitment actions. Compared with fully leaderless coordination, this design reduces negotiation overhead and decision inconsistency, thereby improving coordination efficiency and execution robustness.

Algorithm 1 Timeout-Triggered Recruitment

```

1: Input: collaborative subtask  $q_c$ , arrived-robot set  $\mathcal{R}_{\text{arr}}$ ,
   leader state-record set  $\mathcal{S}_l(\tau_n)$ , free-market location  $x_{\text{free}}$ ,
   free-market robot set  $\mathcal{R}_f$ , reachable robot set  $\mathcal{R}_b$ 
2: if  $|\mathcal{R}_{\text{arr}}| < n_{\text{req}}(q_c)$  then
3:   Construct  $\mathcal{R}_{\text{fit}}^l$  from robots in  $\mathcal{R}_b$  that are available and
   type-compatible with the missing roles of  $q_c$ .
4:   if  $|\mathcal{R}_{\text{arr}} \cup \mathcal{R}_{\text{fit}}^l| \geq n_{\text{req}}(q_c)$  then
5:     Recruit the required robots from  $\mathcal{R}_{\text{fit}}^l$ .
6:     Update  $T_{\text{wait}}$  via Eq. (13).
7:     if waiting time  $t < T_{\text{wait}}$  then
8:       Leader  $r_l$  broadcasts a start signal for subtask  $q_c$ .
9:     else
10:      Leader  $r_l$  announces failure of subtask  $q_c$ .
11:    end if
12:  end if
13:  Construct  $\mathcal{R}_{\text{fit}}^f$  from robots in  $\mathcal{R}_f$  that are type-
   compatible with the remaining missing roles of  $q_c$ .
14:  if  $|\mathcal{R}_{\text{arr}} \cup \mathcal{R}_{\text{fit}}^l \cup \mathcal{R}_{\text{fit}}^f| < n_{\text{req}}(q_c)$  then
15:    Leader  $r_l$  announces failure of subtask  $q_c$ .
16:  end if
17:  Extract  $\{x_p(\tau_n) \mid r_p \in \mathcal{R}_{\text{arr}}\}$  from  $\mathcal{S}_l(\tau_n)$ .
18:   $r_p \leftarrow \arg \min_{r_p \in \mathcal{R}_{\text{arr}}} d(x_p(\tau_n), x_{\text{free}})$ 
19:  Recruit robots in  $\mathcal{R}_{\text{fit}}^l$  and dispatch  $r_p$  to the free market.
20:  Compute timeout  $T_{\text{leave}}^p$  via Eq. (15).
21:  if recruitment succeeds then
22:    Leader  $r_l$  broadcasts a start signal for subtask  $q_c$ .
23:  else if waiting time  $t \geq T_{\text{leave}}^p$  or recruitment fails then
24:    Leader  $r_l$  announces failure of subtask  $q_c$ .
25:  end if
26: end if

```

Upon arriving at a subtask region, robot r_k broadcasts an arrival signal $e_k = \{t_e^k, q_k, a_k\}$, where t_e^k is the arrival timestamp at the task region, and q_k and a_k denote the current subtask and the action performed by r_k , respectively. The robot that arrives first at the task region is selected as leader r_l to prevent conflicts caused by asynchronous arrivals. Let q_c denote the collaborative subtask currently coordinated by leader r_l . If the number of robots that arrive within the collab-

oration waiting time T_{wait} satisfies the requirement of q_c , the leader broadcasts a start signal and the robots begin executing the collaborative subtask. Otherwise, once the waiting time exceeds T_{wait} , timeout-triggered recruitment is initiated.

The collaboration waiting time T_{wait} is dynamically determined using leader r_l 's robot-state information set $S_l(\tau_n)$. It is defined below in terms of the earliest arrival time T_{early} and the latest arrival time T_{late} .

$$T_{\text{wait}} = \left(1 + \beta_w \frac{n_{\text{req}}(q_c)}{n_{\text{req}}^{\max}}\right) (T_{\text{late}} - T_{\text{early}}) \quad (13)$$

where β_w is an adjustment coefficient. In this paper, β_w is set to 1.25 based on experiments. Here, $n_{\text{req}}(q_c)$ denotes the number of robots required for subtask q_c , and $n_{\text{req}}^{\max} = \max_{q_j \in Q} n_{\text{req}}(q_j)$ denotes the maximum robot requirement across all subtasks. This design uses the arrival-time spread $(T_{\text{late}} - T_{\text{early}})$ as the baseline waiting window and scales it by $n_{\text{req}}(q_c)/n_{\text{req}}^{\max}$, so subtasks requiring more robots are given a proportionally longer waiting time to improve collaboration success while avoiding excessive delay for smaller teams. The rationale is that larger teams are harder to synchronize under limited communication: more robots imply greater arrival-time variability, so the probability that all required robots arrive within a short window is lower.

When leader r_l observes that the collaboration waiting time has reached T_{wait} and the number of arrived robots is still below the required cardinality for q_c , timeout-triggered recruitment is activated. The procedure is summarized in Algorithm 1. The leader first performs local recovery. It scans the reachable robot set $\mathcal{R}_b$, which consists of robots currently discoverable by r_l through direct links or multi-hop relays, and extracts robots that are both available and type-compatible with the missing roles of q_c . Here, available robots are those that are not executing other subtasks, are not already committed to recruitment, and have not been recruited by other robots. This locally feasible set is denoted by $\mathcal{R}_{\text{fit}}^l$. If the union of $\mathcal{R}_{\text{arr}}$ and $\mathcal{R}_{\text{fit}}^l$ is sufficient to satisfy the requirement of q_c , the leader recruits the required robots from $\mathcal{R}_{\text{fit}}^l$. The recruitment message includes the subtask ID of q_c , missing number and type requirements, and publication timestamp τ_n .

During recruitment, leader r_l broadcasts recruitment messages and follows a first-come-first-served strategy among received bids. Bidder $r_b \in \mathcal{R}_{\text{fit}}^l$ applies according to message publication time. Upon receiving a bid from robot $r_b \in \mathcal{R}_{\text{fit}}^l$, leader r_l sends a lock request $I_{\text{lock}}^l = \{q_c, \text{ID}_b\}$ to r_b , where ID_b is the ID of r_b . The unlocked bidder r_b replies with $I_{\text{agree}}^b = \{q_c, \text{ID}_b\}$ and becomes locked. After successful recruitment, r_l broadcasts $I_{\text{finish}}^l = \{q_c\}$, and the locked robots proceed to q_c . Leader r_l continues broadcasting recruitment messages until the required robots are obtained or the recruitment exceeds T_{bid} . The collaboration waiting time T_{wait} is then updated according to the predicted arrival times of the recruited robots. If the updated waiting bound is still exceeded before team formation is completed, the leader declares the subtask failed. The bidding timeout T_{bid} is defined below.

$$T_{\text{bid}} = |\mathcal{R}| \frac{1}{f_{\text{com}}} N_{\text{round}} + t_{\text{comp}}, \quad (14)$$

where $|\mathcal{R}|$ is the total number of robots, N_{round} is the number of recruiter–bidder message rounds required by the recruitment process, and t_{comp} is the computation time. In this paper, N_{round} is set to 5 and t_{comp} is set to 2.0 s.

If local recovery is insufficient, the leader expands the search to the free market. It first constructs a backup candidate set $\mathcal{R}_{\text{fit}}^f$ from robots in $\mathcal{R}_f$ whose capabilities match the remaining missing roles of q_c . If the union of the arrived robots $\mathcal{R}_{\text{arr}}$, the locally reachable candidates $\mathcal{R}_{\text{fit}}^l$, and the free-market candidates $\mathcal{R}_{\text{fit}}^f$ is still insufficient, the subtask is immediately treated as failed. Otherwise, the leader selects a recruiter r_p from the arrived robots according to

$$r_p = \arg \min_{r \in \mathcal{R}_{\text{arr}}} d(x_r(\tau_n), x_{\text{free}}),$$

where $x_r(\tau_n)$ is the position of robot r recorded in $S_l(\tau_n)$. The recruitment procedure executed by recruiter r_p in the free market is the same as that used by leader r_l . Leader r_l declares subtask failure if recruitment is unsuccessful within T_{leave}^p . The leave timeout is defined below.

$$T_{\text{leave}}^p = \left(1 + \zeta \frac{d_{\text{leave}}}{d_{\text{max}}}\right) \left(\frac{2d_{\text{leave}}}{v_p}\right) + T_{\text{bid}} \quad (15)$$

where d_{max} is the maximum inter-target-region distance, and ζ is an adjustment coefficient. Based on experiments, ζ is set to 0.75 in this paper. d_{leave} denotes the distance from recruiter r_p to the free market. By adapting T_{leave}^p to travel distance, the leader avoids declaring failure too early for distant recruiters and avoids excessive waiting for nearby recruiters. The first term in Eq. (15) estimates the round-trip travel time between the subtask region and the free market, while T_{bid} reserves time for recruitment in the market.

If recruitment is still unsuccessful within T_{leave}^p , subtask q_c is declared failed, and the leader notifies all robots currently executing q_c . Failed subtasks are added to each robot's failed-subtask set and are temporarily skipped until robots enter the free market.

3) *Failure Handling for Collaborative Subtasks*: Failed collaborative subtasks are handled through free-market-based reassignment. Notably, the communication network among robots in the free market is assumed to be connected. Once a robot enters the free market, failed subtasks are reallocated through a recruiter–bidder mechanism after state information is synchronized, enabling robots to recover from collaboration failures using updated information.

When robot r_k finds that its current optional-subtask set contains only failed subtasks, it enters the free market for reassignment. After arriving at the free market, r_k first synchronizes recruitment messages and state information, and then checks whether it can serve as a recruiter. Let $\mathcal{F}_k$ denote the failed-subtask set maintained by r_k . Let $\mathcal{R}_{\text{rec}}$ denote the current recruiter set in the free market, and let $\mathcal{R}_k^< \subseteq \mathcal{R}_{\text{rec}}$ denote the recruiters that published earlier than r_k . Robot r_k becomes a recruiter only if it has not received any valid recruitment message for a compatible subtask. In that case, it selects one failed subtask from $\mathcal{F}_k$ according to the shortfall-and-distance rule and publishes recruitment message for subtask.

Let $n_{\text{rem}}^k(t, j)$ represent the remaining number of type- j robots that are available to recruiter r_k at time t after accounting for earlier recruitment requests. For each failed subtask $q \in \mathcal{F}_k$, the shortfall is defined below. $\Delta(q) = \sum_{j \in \mathcal{T}} \max(0, n_{\text{req}}(q, j) - n_{\text{rem}}^k(t, j))$. Recruiter r_k then selects the failed subtask with the minimum shortfall. The selected subtask is defined below.

$$q_k^* = \arg \min_{q \in \mathcal{F}_k} \Delta(q). \quad (16)$$

If multiple candidates share the same minimum shortfall, r_k selects the one whose target region is nearest. After determining the subtask to be handled first, r_k publishes a recruitment message $I_{\text{bid}}^k = \{\tau_k, r_k, q_k^*, \eta_k\}$. Here, τ_k is the publishing time, and η_k is the type-cardinality requirement vector of the recruitment message.

After recruitment messages are published, any robot in the free market that is not acting as a recruiter is treated as a bidder. Bidder r_c selects a recruitment message according to task priority and message publishing time. Let $\mathcal{M}_c(\tau_n)$ be the set of active recruitment messages received by bidder r_c at time τ_n . The compatible message set is defined as $\mathcal{M}_c^{\text{com}}(\tau_n) = \{I_{\text{bid}}^k \in \mathcal{M}_c(\tau_n) \mid r_c \text{ is compatible with } q_k^*\}$. For each message I_{bid}^k , let $\rho(q_k^*)$ denote the priority rank of subtask q_k^* , where a smaller value means higher priority. The bidder-selected recruitment message is defined as $I_c^* = \arg \min_{I_{\text{bid}}^k \in \mathcal{M}_c^{\text{com}}(\tau_n)} (\rho(q_k^*), \tau_k)$, where minimization is lexicographic: bidders first select the highest-priority subtask, and among subtasks with the same priority, they choose the earliest published recruitment message. In this paper, high-priority subtasks refer to emergency subtasks issued by leader-dispatched recruiters, whereas low-priority subtasks refer to failed or currently unassigned subtasks handled through free-market reassignment.

During recruitment, recruiter r_k may receive a compatible recruitment message I_{bid}^h and become a potential bidder. If the currently available free-market robots are insufficient to satisfy both recruitments simultaneously, r_k compares $(\rho(q_h^*), \tau_h)$ with $(\rho(q_k^*), \tau_k)$. If $\rho(q_h^*) < \rho(q_k^*)$, or if $\rho(q_h^*) = \rho(q_k^*)$ and $\tau_h < \tau_k$, then r_k relinquishes its recruiter role, cancels its own recruitment, and switches to bidder mode for I_{bid}^h .

Recruiter r_k follows the same first-come-first-served rule described in Sec. VI-C2. Let $\mathcal{L}_k$ denote the bidder set currently locked by recruiter r_k . Upon receiving a bid from robot r_c , recruiter r_k sends a lock request $I_{\text{lock}}^k = \{q_k^*, \text{ID}_c\}$ to r_c , where q_k^* is the recruited subtask and ID_c is the ID of r_c . The uncommitted bidder r_c replies with $I_{\text{agree}}^c = \{q_k^*, \text{ID}_c\}$ and becomes locked. Once recruitment is completed, r_k broadcasts $I_{\text{finish}}^k = \{q_k^*\}$, and locked robots then proceed to execute q_k^* .

A bidder is unlocked if it receives no message from recruiter r_k within T_{send} seconds, which indicates a presumed recruiter failure, or if a higher-priority subtask arrives. The unlocking timeout is defined below.

$$T_{\text{send}} = \left(1 + \left\lceil \frac{d_{c,k}}{D} \right\rceil\right) \frac{1}{f_{\text{com}}}, \quad (17)$$

where $d_{c,k}$ is the distance from bidder r_c to recruiter r_k , D is the communication range, and f_{com} is the communication fre-

quency. In such cases, bidder r_c transmits $I_{\text{unlock}}^c = \{q_k^*, \text{ID}_c\}$ to prompt r_k to remove it from the locked-robot list $\mathcal{L}_k$.

D. Patrol Mechanism for Task Blocking

Temporal logic constraints can cause blocking while robots wait for prerequisite subtasks to be confirmed. For example, if subtask “inspect Zone B” is constrained to start only after “verify tunnel entrance” is completed, a robot assigned to Zone B may remain idle when the completion message from the entrance team cannot be received in time under limited communication. To address this issue, a patrol mechanism is proposed to proactively gather and relay prerequisite status information, thereby releasing blocked subtasks earlier. Let $\mathcal{Q}_k \subseteq \mathcal{Q}$ denote the set of subtasks executable by robot r_k . For each $q_i \in \mathcal{Q}_k$, let $\mathcal{P}_i$ denote the prerequisite-subtask set of q_i . For each prerequisite subtask $q_j \in \mathcal{P}_i$, robot r_k maintains an estimated completion time $\hat{t}_k(q_j)$. The estimate $\hat{t}_k(q_j)$ is computed in the same form as $T_{\text{late}}^{k,m}$ in Sec. VI-B2. For subtasks constrained only by “do not start before the prerequisite starts”, the same estimation logic is used, except that the prerequisite start time is estimated by subtracting the prerequisite execution duration from $\hat{t}_k(q_j)$.

A robot triggers the patrol mechanism only when two conditions are simultaneously satisfied: i) there is no currently unassigned optional subtask that can be executed immediately, and ii) for at least one pending subtask q_i , the current time exceeds the latest estimated completion time of its prerequisites, i.e., $\tau_{\text{now}} > \max_{q_j \in \mathcal{P}_i} \hat{t}_k(q_j)$. This means the robot has waited beyond the expected prerequisite completion horizon but still cannot confirm prerequisite satisfaction from local information. In this case, the robot starts patrol to actively collect missing status updates. The initial patrol targets include all regions associated with unresolved prerequisite subtasks and the free market. During patrol, the target set is refined as follows: i) remove regions whose prerequisites are already completed or reliably verified, ii) skip prerequisites whose successor subtasks have already started, iii) omit prerequisites whose own predecessor conditions are still unmet, and iv) update optional-subtask candidates and their estimated completion times using newly gathered information. Once at least one executable optional subtask is identified, the robot exits patrol and switches to task execution.

After patrol, if the estimated completion time is updated, robots will re-patrol based on new timings; otherwise, robots will wait t_p seconds before the next patrol. The patrol interval is defined below.

$$t_p = \max_{q_i \in \mathcal{Q}_k} \max_{q_j \in \mathcal{P}_i} \left(\frac{d_f^j}{v_a} + \max_{a_\ell \in \text{LA}(q_j)} \text{LC}(a_\ell) \right) \quad (18)$$

where d_f^j is the free-market-to-subtask distance. t_p assumes the subtask has failed and will be reassigned in the free market.

VII. ALGORITHM ANALYSIS

This section proves that, under the assumptions in Sec. III-B and a finite subtask set (i.e., $|\mathcal{Q}| < \infty$), the proposed algorithm guarantees task completion.

Theorem 1: Completion of unfinished low-priority subtasks handled through free-market reassignment is guaranteed.

Proof: Consider an unfinished low-priority subtask q . For each type index $j \in \mathcal{T}$, let $n_{\text{req}}(q, j)$ denote the required number of type- j robots, and let $n_{\text{free}}(t', j)$ denote the number of type- j robots in the free market at time t' . Take any robot r_k that is capable of executing q . We first show that r_k enters the free market within a finite time T_{bound}^k . For q , r_k falls into one of three cases: i) $q \in \mathcal{F}_k$; ii) the currently available teammates are insufficient; iii) q has been assigned to other robots. If q is non-collaborative, case iii) reduces to case ii) after at most $T_{\text{late}}^k(q)$. Moreover, $T_{\text{late}}^k(q) \leq T_{\text{timeout}}^{\text{max}} \leq \alpha_{\text{late}}(1 + \lceil \frac{d_{\text{max}}}{D} \rceil) \max\left(T_{\text{enc}}, \frac{1}{f_{\text{com}}}\right) + t_s^{\text{max}} + L_{\text{task}}^{\text{max}}$ according to Eq. (5) and Eq. (4), where d_{max} is the maximum workspace distance, t_s^{max} is the latest estimated subtask start time in $\mathcal{S}_k(\tau_n)$, and $L_{\text{task}}^{\text{max}} = \max_{q_j \in \mathcal{Q}} \max_{a_i \in \text{LA}(q_j)} \text{LC}(a_i)$. Assume that the maximum execution time of a subtask for r_k is $T_{\text{task}}^{\text{max}, k}$. The subtask execution time is finite, and co-safe LTL implies $|\mathcal{Q}| < \infty$. Therefore, robot r_k enters the free market within a finite time satisfying

$$T_{\text{bound}}^k \leq |\mathcal{Q}|(T_{\text{task}}^{\text{max}, k} + T_{\text{timeout}}^{\text{max}}).$$

Now consider three cases for q . i) Recruitment proceeds without interruption by high-priority subtasks. According to assumption iii) and the condition that robots do not issue recruitment requests that exhaust free-market resources, all robots capable of q enter the free market within a finite time. Therefore, there exists a time $t' \in (t_0, \infty)$ such that

$$\forall j \in \mathcal{T}, \quad n_{\text{free}}(t', j) \geq n_{\text{req}}(q, j).$$

Hence, q can be successfully allocated. ii) The recruitment process is interrupted by emergency-subtask recruitment. Since $|\mathcal{Q}| < \infty$ and any departing robot r_k is guaranteed to rejoin the free market within T_{bound}^k , interruptions terminate in finite steps, which reduces to case i). iii) If q remains unposted, it is because all capable robots are already engaged in earlier recruitments. Since every existing recruitment falls into case i) or ii), q will eventually be posted. Hence, in all cases, q will eventually be posted and successfully allocated.

Theorem 2: The proposed algorithm guarantees all subtasks will be completed under assumptions ii) and iii).

Proof: The feasibility of CBTA is discussed in [5]. Under assumption iii), incomplete subtasks arise from: i) missing prerequisite subtask information, ii) insufficient teammates, or iii) expected robots fail to reach the target region. Case i) is resolved by the patrol mechanism. In cases ii) and iii), insufficient teammates and subtask failures eventually lead robots to enter the free market, since $|\mathcal{Q}| < \infty$. By Theorem 1, all subtasks will ultimately be completed.

Theorem 3: The proposed algorithm is complete. That is, if there exists a feasible plan satisfying φ , then the proposed algorithm is guaranteed to find a plan that satisfies φ .

Proof: The key idea in the proof is to first show that feasible words are preserved after automaton pruning and then use this fact to establish the existence of feasible paths, task allocations, and executable robot trajectories. The detailed proof follows the sequence of feasibility arguments for the

DFS-based path search, the task decomposition, CBTA task allocation, and path planning for each assigned subtask.

Specifically, the automaton pruning step removes only infeasible transitions, and by Lemma 1, any word accepted by the original NBA $\mathcal{B}$ has a corresponding word accepted by the pruned automaton $\mathcal{B}^-$. Therefore, feasible words are preserved. Next, since DFS is complete over finite graphs, an accepting path in $\mathcal{B}^-$ will be discovered. The accepting path is then decomposed into a sequence of subtasks, which by construction respects the temporal ordering encoded in φ . Given the existence of a feasible path that satisfies all subtasks and sufficient time, the CBTA-based allocation procedure is guaranteed to find a feasible assignment.

If a subtask execution fails, Theorems 1 and 2 ensure that the subtask can be reassigned and successfully executed by another robot, guaranteeing completion of the overall plan. During execution, feasible path planning for each assigned subtask exists, as all transitions in $\mathcal{B}^-$ correspond to realizable motions in the workspace.

VIII. RESULTS

The algorithm is evaluated through simulations and physical experiments. LTL formulas are converted to NBAs using LTL2BA [38]. Simulations are run in Python 3 on an 11th Gen Intel® Core™ i7-11390H with 16 GB RAM. Physical experiments are conducted on Ubuntu 18.04 with ROS.

A. Numerical Simulation

We evaluate the proposed framework under communication-constrained settings, with a focus on its effectiveness in handling task blocking, dynamic robot changes, collaborative coordination, and failure recovery. To this end, the proposed framework is examined through four test scenarios:

i) Test 1 evaluates the patrol mechanism through an ablation study. For each instance, the patrol mechanism is enabled or disabled under the same task setting, while the communication range is varied to examine how efficiently blocked prerequisite subtasks are resolved and how this affects the final task completion rate.

ii) Test 2 compares timeout-triggered recruitment with timeout-based local recruitment [33] under multiple communication ranges. This comparison is used to evaluate the impact of timeout-triggered recruitment on collaboration success and overall completion efficiency.

iii) Test 3 evaluates free-market-based reassignment in preventing repeated subtask failures, using failure-to-tail reassignment as a baseline with $D = 1.5$ m. The goal is to examine whether free-market-based reassignment can reduce repeated failure chains and improve recovery stability under limited communication.

iv) Test 4 evaluates the adaptability of the proposed framework to dynamic robot changes (e.g., robots joining or temporarily becoming unavailable during execution). We evaluate whether the framework can maintain feasible allocation updates and continue task progress without global replanning.

The experimental setup consists of 12 target regions, denoted by $\mathcal{L} = \{l_1, l_2, \dots, l_{12}\}$, and a 12-robot team comprising

three heterogeneous robot types: type-1, type-2, and type-3. The spatial arrangement of regions and robot deployment follows Fig. 3. To evaluate robustness under different task-location assignments, we treat the region labels $l_1 \sim l_{12}$ as permutable and generate 25 randomized benchmark instances (Inst. 1–Inst. 25) by varying the label-to-location mapping while keeping the underlying workspace geometry unchanged.



Fig. 3. Simulation environment for Tests 1–4. The 30×15 simulation environment contains 12 target regions $l_1 \sim l_{12}$ and 12 robots $r_1 \sim r_{12}$.

1) *Test 1*: Consider the LTL-specified global task $\varphi = \Diamond((l_{2,1}^{12} \wedge l_{1,1}^{12}) \wedge \Diamond((l_{2,2}^{12} \wedge \neg(l_{2,1}^{12} \wedge l_{1,1}^{12})) \wedge \Diamond(l_{2,1}^{6} \wedge \neg l_{2,2}^{7}))) \wedge \Diamond l_{3,1}^2 \wedge \Diamond l_{3,1}^3 \wedge \Diamond(l_{3,3}^9 \wedge \neg p_4)$. After preprocessing φ , the resulting subtask set is $\Omega_\varphi = \{q_1, q_2, q_3, q_4, q_5, q_6\}$, where $q_1 = \{1, \{l_{3,3}^9\}, \{p_4\}\}$, $q_2 = \{2, \{l_{3,1}^3\}, \{\}\}$, $q_3 = \{3, \{l_{2,1}^2\}, \{\}\}$, $q_4 = \{4, \{l_{2,1}^{12}, l_{1,1}^{12}\}, \{\}\}$, $q_5 = \{5, \{l_{2,2}^7\}, \{\}\}$, $q_6 = \{6, \{l_{2,1}^6\}, \{\}\}$. The objective of this experiment is to verify the causal role of the patrol mechanism in resolving task blocking under limited communication. To this end, we evaluate Inst. 1–Inst. 25 under two settings: Case 1 (with patrol) and Case 2 (without patrol), while keeping task instances, robot settings, and communication ranges identical. The only difference is whether robots are allowed to proactively gather information.

As shown in Fig. 4, when communication range decreases, Case 2 exhibits a significant decline in task completion rate because blocked robots cannot reliably obtain prerequisite updates and therefore remain in waiting states for long periods. Notably, Case 2 can still complete some instances at small communication ranges when a blocked robot happens to encounter another robot that already carries the required prerequisite information; however, this information-recovery path is opportunistic and unstable, and its success strongly depends on workspace geometry, robot trajectories, and task-region distribution. In contrast, Case 1 maintains a 100% completion rate across all tested ranges: patrol converts passive waiting into active information acquisition, closes prerequisite-information gaps in a structured way, and enables blocked subtasks to be released in time without relying on chance encounters. These results highlight the key advantage of the patrol mechanism: it provides robust information recovery under communication constraints, rather than probabilistic recovery determined by incidental robot meetings.

2) *Test 2*: Considering the same global task φ as in Test 1, we further evaluate timeout-triggered recruitment. Timeout-triggered recruitment (TTR) is compared with timeout-based local recruitment (TLR) [33]. The two methods differ only in how replacement candidates are searched after timeout: TLR selects from robots currently reachable by the leader (usually the nearest one), while TTR can dispatch a recruiter to the free market to obtain additional qualified robots outside the

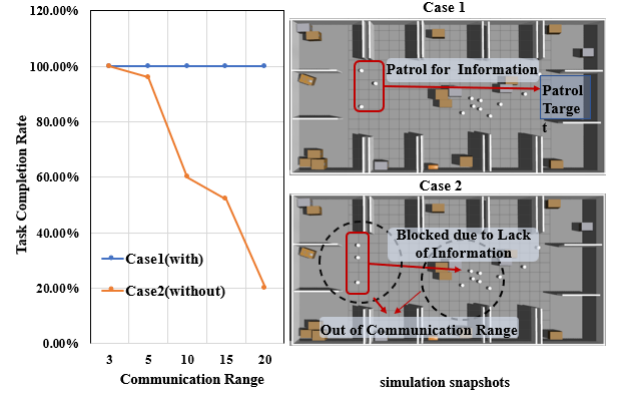


Fig. 4. The results of Test 1. Left: task completion rates under different communication ranges. Right: simulation snapshots and explanation of the differences between Case 1 (with patrol) and Case 2 (without patrol).

local communication neighborhood. In each run, one robot is randomly removed during execution of q_1 to create the same failure trigger for both methods. Performance is measured by the completion time of type-3-related subtasks (T3-subtask time) under different communication ranges. We use this metric instead of total mission time because it directly captures recovery efficiency for the affected collaborative chain, while total mission time can be perturbed by unrelated subtasks.

Fig. 5 and Table I show that TTR yields shorter T3-subtask time in most instances, with the largest gains at small communication ranges. The reason is that, under sparse connectivity, TLR is constrained by local visibility and may not find a feasible replacement quickly, whereas TTR expands the search scope through the free market and therefore restores collaboration earlier. When communication range is large, the free market is effectively inside the connected neighborhood, so TTR and TLR have similar candidate sets and similar performance. The exceptions (e.g., Inst.2 and Inst.6) are also explainable: both methods behave similarly because q_2 and q_3 are far from initial robot positions and no suitable type-3 robot is available in the free market at timeout; here the dominant bottleneck is temporary resource scarcity, not recruitment strategy. Overall, the experiment indicates that TTR improves robustness mainly by enlarging the effective replacement-search region under communication constraints.

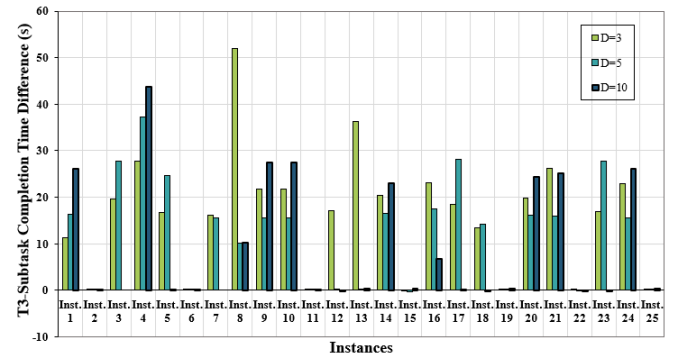


Fig. 5. Comparison between TTR(proposed) and TLR. The positive values indicating superior performance of TTR.

TABLE I
T3-SUBTASK TIME GAP BETWEEN TTR AND TLR UNDER DIFFERENT COMMUNICATION RANGES. ALL TIME GAPS ARE IN SECONDS (S).

D (m)	Inst. 1	Inst. 2	Inst. 3	Inst. 4	Inst. 5	Inst. 6	Inst. 7	Inst. 8	Inst. 9	Inst. 10	Inst. 11	Inst. 12	Inst. 13
3	11.29	0.03	19.56	27.82	16.75	0.12	16.22	52.07	21.69	21.69	0.32	17.09	36.36
5	16.26	0.12	27.73	37.31	24.61	0.23	15.48	10.20	15.49	15.49	0.12	0.32	0.32
10	26.13	0.02	0.00	43.70	0.11	0.09	0.00	10.28	27.52	27.52	0.22	-0.25	0.32

Table I (continued)

D (m)	Inst. 14	Inst. 15	Inst. 16	Inst. 17	Inst. 18	Inst. 19	Inst. 20	Inst. 21	Inst. 22	Inst. 23	Inst. 24	Inst. 25
3	20.43	-0.19	23.10	18.41	13.53	0.32	19.91	26.17	0.27	16.93	22.85	0.12
5	16.57	-0.28	17.49	28.18	14.14	0.09	16.11	16.03	-0.13	27.78	15.56	0.09
10	23.12	0.32	6.74	0.18	-0.24	0.27	24.31	25.08	-0.25	-0.02	26.04	0.27

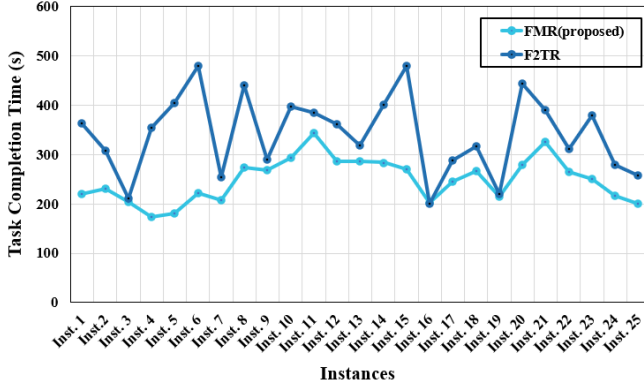


Fig. 6. Comparison between FMR (proposed) and F2TR.

3) *Test 3*: In scenarios with multiple collaborative subtasks under limited communication, repeated failures can propagate across temporally dependent tasks and substantially delay mission completion. This test evaluates whether free-market-based reassignment (FMR) can reduce such cascading delays compared with failure-to-tail reassignment (F2TR), where each failed subtask is appended to the end of the local plan and retried later. The comparison is conducted over Inst. 1–Inst. 25 under $D = 1.5$ m, using task completion time as the primary metric. Consider the LTL-specified task $\varphi_1 = \Diamond l_{1,2}^{11} \wedge \Diamond l_{1,2}^{7,2} \wedge \Diamond l_{1,2}^{2,2} \wedge \Diamond ((l_{2,1}^{2,1} \wedge \neg l_{2,2}^{9,2}) \wedge \Diamond l_{2,2}^{9,2})$. Let $\Omega_\varphi = \{q_1, q_2, q_3, q_4, q_5\}$, where $q_1 = \{1, \{l_{2,1}^{2,1}\}\}$, $q_2 = \{2, \{l_{2,2}^{9,2}\}\}$, $q_3 = \{3, \{l_{1,2}^{2,2}\}\}$, $q_4 = \{4, \{l_{1,2}^{7,2}\}\}$, and $q_5 = \{5, \{l_{1,2}^{11}\}\}$. Five robots are deployed: r_1, r_2, r_3 (type 1) and r_4, r_5 (type 2).

To trigger representative collaborative failures, one type-1 robot is removed before the second (in execution order) type-1 collaborative subtask is completed. This setting is critical because the team has minimal redundancy: a single removal can break feasibility of the current collaboration and force reallocation. Under F2TR, this often leads to deferral and repeated waiting in downstream subtasks. In contrast, FMR synchronizes failure information in the free market and reallocates with a wider, updated candidate set, which is expected to reduce repeated failures and shorten recovery latency. From a mechanism perspective, the key difference is whether recovery decisions are made with fragmented local views (F2TR) or with synchronized team-state information (FMR).

As shown in Fig. 6, FMR generally outperforms F2TR across most instances, indicating that the benefit is systematic

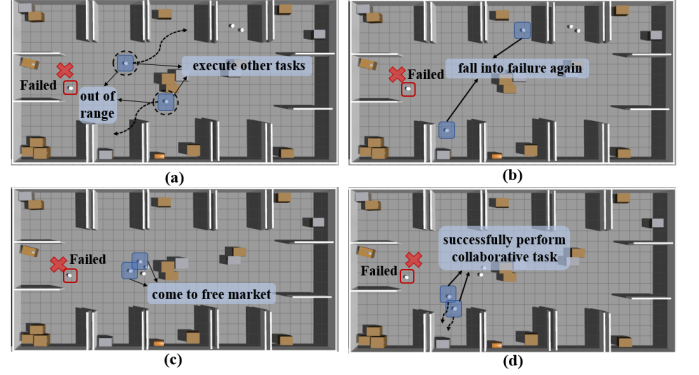


Fig. 7. Schematic comparison of collaborative-failure handling processes in F2TR and the proposed method. Subfigures (a) and (b) illustrate the F2TR process, while (c) and (d) illustrate the process of the proposed method.

rather than case-specific. As shown in Fig. 7, the key reason is that, under dense collaborative dependencies and limited communication, a single failure often creates a “failure–delay–reassignment” loop: delayed recovery of one subtask postpones its successors, which increases the probability of additional waiting and repeated failures. F2TR tends to amplify this loop because failed subtasks are deferred to the tail and revisited with fragmented local information. In contrast, FMR interrupts the loop by synchronizing failure states in the free market and reallocating from a broader, updated candidate set, which shortens recovery latency and reduces repeated failure.

The smaller gaps in instances such as Inst.3 and Inst.16 are also informative: when spatial layout and timing naturally permit sufficient information exchange, even local-tail recovery can approach the performance of FMR. This suggests that FMR’s main advantage appears in communication-stressed or coordination-dense cases. Overall, the results support that FMR improves not only average completion efficiency but also resilience to chained collaborative-subtask disruptions.

4) *Test 4*: At $D = 10$ m, with regions $l_1 \sim l_{12}$ arranged as shown in Fig. 3, Test 4 evaluates the proposed framework under dynamic changes in team composition by introducing both robot addition and robot removal during task execution. At $t = 15$, a new type-3 robot, r_{13} , joins the system. After entering the communication range and synchronizing with the team, r_{13} is incorporated into the candidate set and assigned to subtask q_2 . This online reallocation updates the task assignment according to the new team composition, thereby reducing the burden on the original team and yielding

a plan suited to the expanded set of available robots.

Later, at $t = 80$, robot r_{10} is removed, leaving collaborative subtask q_1 understaffed. In response, the leader robot r_{11} keeps the remaining participants in a waiting state while checking whether the missing role can be filled locally. Once the waiting time exceeds T_{wait} , r_{11} activates the timeout-based recovery mechanism and dispatches r_9 as a recruiter to seek a qualified replacement. After recruitment succeeds, the team is re-formed, and q_1 is eventually completed by r_{11} , r_{13} , and r_9 , as shown in Fig. 8.

From a process perspective, this scenario captures the full evolution from disturbance to response and eventual recovery. Specifically, the removal of r_{10} first introduces the risk of collaborative-subtask failure due to an insufficient number of participants for q_1 ; the timeout-triggered recruitment mechanism then restores the required team size; and finally, task execution returns to a consistent and stable state. Therefore, the trajectory shown in Fig. 8 demonstrates not only successful task completion, but also the effectiveness of the proposed framework in supporting both collaborative-failure recovery and online task reallocation under dynamic team changes.

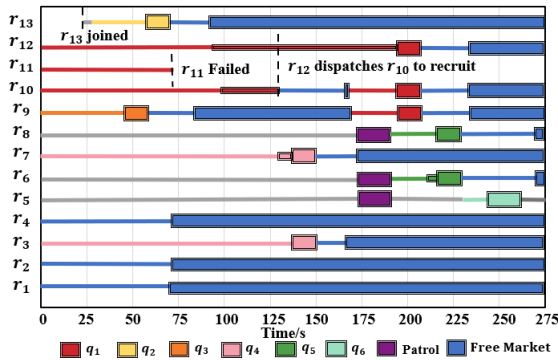


Fig. 8. Task timelines in Test 4. Wide blocks denote subtasks, narrow colored blocks indicate waiting. Gray lines mean no subtask and not in the free market, and other colored lines represent traveling toward the target region.

B. Physical Experiment

Consider the environment depicted in Fig. 9(a), where l_1 denotes the sorting area, l_2 the storage center, and l_3 the monitoring room. Four robots of three distinct types operate with a 2.0-meter communication range. The mission specification $\varphi = \Diamond(l_{1,1}^1 \wedge l_{3,1}^1 \wedge l_{2,1}^1) \wedge \Diamond l_{3,1}^2 \wedge \Diamond l_{1,1}^3 \wedge \Diamond l_{2,1}^2 \wedge (\neg l_{2,1}^2 \mathcal{U} l_{1,1}^4) \wedge (\neg l_{1,1}^3 \mathcal{U} (l_{1,1}^1 \wedge l_{3,1}^1 \wedge l_{2,1}^1))$ requires one robot of each type (1, 2, and 3) to inspect, classify, and process goods in the sorting area l_1 ; a type-3 robot to inspect the storage area l_2 ; and, upon completion of the sorting tasks, a type-1 robot to report in the monitoring room l_3 , after which a type-2 robot processes goods in the storage area l_2 .

Experimental results (Fig. 9) show that the proposed mechanisms are effective in a real communication-limited setting. First, the collaborative operation in l_1 is completed before the dependent actions in l_3 , which is consistent with the temporal-order constraints in φ and indicates that prerequisite information is propagated in time to avoid logical blocking. Second, after leaving l_1 , robot r_1 proceeds to l_3 while r_2

is temporarily outside direct 2.0-meter communication range; nevertheless, team coordination is preserved through free-market-mediated information exchange, so execution does not stall due to link interruption. Third, once the l_3 -related subtask is confirmed complete, r_2 transitions to l_2 to execute the subsequent dependent task, showing that task handoff remains consistent even across intermittent connectivity. Overall, the physical experiment validates three key properties in combination: i) correctness of temporally constrained execution, ii) communication resilience via the free market under range limitations, and iii) continuity of distributed task progression without centralized global replanning.

IX. CONCLUSIONS

This paper presents a distributed task allocation framework with resilient task coordination for heterogeneous multi-robot systems under temporal logic and communication constraints. Starting from a global sc-LTL specification, the framework derives executable subtasks through automaton pruning and task decomposition, and generates initial plans using CBTA. To sustain task progress under limited communication, the framework incorporates two complementary coordination mechanisms: a patrol mechanism for resolving task blocking through active information gathering, and a coordination mechanism for collaborative subtasks that combines leader-based coordination, timeout-triggered recruitment, and free-market-based reassignment. Theoretical analysis establishes task-completion guarantees under the assumptions of this paper. Simulations on 25 benchmark instances and a physical experiment demonstrate that the proposed framework improves the robustness of distributed task coordination under incomplete and delayed team-state information.

REFERENCES

- [1] Y. Qu, B. Xin, Q. Wang, R. Li, and Z. Du, "A novel memetic algorithm for distributed shape formation of swarm robots with both acceleration and velocity constraints," *Science China Information Sciences*, vol. 67, no. 12, p. 222201, 2024.
- [2] H. Fang, S. Wei, Y. Shi, and J. Ou, "Multi-agent ergodic surveillance under unknown target distribution," *Unmanned Systems*, 2025, doi: 10.1142/S2301385025440054.
- [3] L. Zhou, M. Jing, L. Zhong, Z. Yu, and X. Zeng, "Task-binding based branch-and-bound algorithm for noc mapping," in *2012 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2012, pp. 648–651.
- [4] T. Okubo and M. Takahashi, "Simultaneous optimization of task allocation and path planning using mixed-integer programming for time and capacity constrained multi-agent pickup and delivery," in *2022 22nd International Conference on Control, Automation and Systems (ICCAS)*. IEEE, 2022, pp. 1088–1093.
- [5] S. Wang, Y. Liu, Y. Qiu, and J. Zhou, "Consensus-based decentralized task allocation for multi-agent systems and simultaneous multi-agent tasks," *IEEE Robotics and Automation Letters*, vol. 7, no. 4, pp. 12 593–12 600, 2022.
- [6] L. Govoni and A. Cristofaro, "A decentralized control and task allocation framework for heterogeneous redundant multiagent systems," *IEEE Transactions on Cybernetics*, 2025.
- [7] Y. Wang, S. Ren, and C. Wang, "Multi-uav multi-task allocation based on cognitive state-based hybrid contract net protocol," in *2025 10th International Conference on Computer and Communication System (ICCCS)*. IEEE, 2025, pp. 921–926.
- [8] A. Maoudj, B. Bouzouia, A. Hentout, A. Kouider, and R. Toumi, "Distributed multi-agent approach based on priority rules and genetic algorithm for tasks scheduling in multi-robot cells," in *IECON 2016-42nd Annual Conference of the IEEE Industrial Electronics Society*. IEEE, 2016, pp. 692–697.

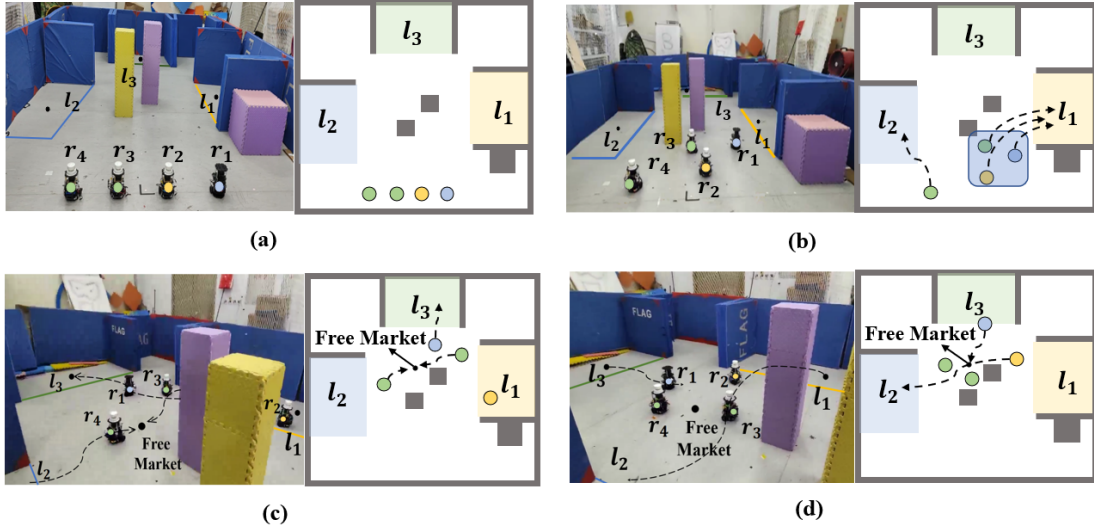


Fig. 9. The dashed lines indicate the robot trajectories. In (a), l_1 to l_3 represent the areas of interest. There are three kinds of robots: $\{r_1\}$, $\{r_2\}$ and $\{r_3, r_4\}$. In (b)-(d), the robots are executing their subtasks.

- [9] J. Xie and C.-C. Liu, "Multi-agent systems and their applications," *Journal of International Council on Electrical Engineering*, vol. 7, no. 1, pp. 188–197, 2017.
- [10] K. Leahy, D. Zhou, C.-I. Vasile, K. Oikonomopoulos, M. Schwager, and C. Belta, "Persistent surveillance for unmanned aerial vehicles subject to charging and temporal logic constraints," *Autonomous Robots*, vol. 40, no. 8, pp. 1363–1378, 2016.
- [11] S. Xu, X. Luo, Y. Huang, L. Leng, R. Liu, and C. Liu, "NI2hlt2plan: Scaling up natural language understanding for multi-robots through hierarchical temporal logic task specifications," *IEEE Robotics and Automation Letters*, pp. 1–8, 2025.
- [12] C. Baier and J.-P. Katoen, *Principles of model checking*. MIT press, 2008.
- [13] L. Li, Z. Chen, H. Wang, and Z. Kan, "Task allocation of heterogeneous robots under temporal logic specifications with inter-task constraints and variable capabilities," *IEEE Transactions on Automation Science and Engineering*, 2025.
- [14] P. Yu and D. V. Dimarogonas, "Distributed motion coordination for multirobot systems under ltl specifications," *IEEE Transactions on Robotics*, vol. 38, no. 2, pp. 1047–1062, 2021.
- [15] S. L. Smith, J. Tümová, C. Belta, and D. Rus, "Optimal path planning under temporal logic constraints," in *2010 IEEE/RSJ International Conference on Intelligent Robots and Systems*. IEEE, 2010, pp. 3288–3293.
- [16] A. Ulusoy, S. L. Smith, X. C. Ding, C. Belta, and D. Rus, "Optimality and robustness in multi-robot path planning with temporal logic constraints," *The International Journal of Robotics Research*, vol. 32, no. 8, pp. 889–911, 2013.
- [17] Y. Kantaros and M. M. Zavlanos, "Sampling-based optimal control synthesis for multirobot systems under global temporal tasks," *IEEE Transactions on Automatic Control*, vol. 64, no. 5, pp. 1916–1931, 2018.
- [18] —, "Stylus*: A temporal logic optimal control synthesis algorithm for large-scale multi-robot systems," *The International Journal of Robotics Research*, vol. 39, no. 7, pp. 812–836, 2020.
- [19] X. Luo and M. M. Zavlanos, "Temporal logic task allocation in heterogeneous multirobot systems," *IEEE Transactions on Robotics*, vol. 38, no. 6, pp. 3602–3621, 2022.
- [20] Z. Chen and Z. Kan, "Real-time reactive task allocation and planning of large heterogeneous multi-robot systems with temporal logic specifications," *The International Journal of Robotics Research*, vol. 44, no. 4, pp. 640–664, 2024.
- [21] P. Schillinger, M. Bürger, and D. V. Dimarogonas, "Decomposition of finite ltl specifications for efficient multi-agent planning," in *Distributed Autonomous Robotic Systems: The 13th International Symposium*. Springer, 2018, pp. 253–267.
- [22] —, "Simultaneous task allocation and planning for temporal logic goals in heterogeneous multi-robot systems," *The International Journal of Robotics Research*, vol. 37, no. 7, pp. 818–838, 2018.
- [23] J. Liu, F. Li, X. Jin, and Y. Tang, "Distributed task allocation for multi-agent systems: A submodular optimization approach," *arXiv preprint arXiv:2412.02146*, 2024.
- [24] Z. Liu, M. Guo, W. Bao, and Z. Li, "Fast and adaptive multi-agent planning under collaborative temporal logic tasks via poset products," *Research*, vol. 7, p. 0337, 2024.
- [25] Y. Zhu *et al.*, "Dexter-llm: Dynamic and explainable coordination of multi-robot systems in unknown environments via large language models," *arXiv preprint arXiv:2508.14387*, 2025.
- [26] I. Filippidis, D. V. Dimarogonas, and K. J. Kyriakopoulos, "Decentralized multi-agent control from local ltl specifications," in *2012 IEEE 51st IEEE Conference on Decision and Control (CDC)*, 2012, pp. 6235–6240.
- [27] M. Guo and D. V. Dimarogonas, "Multi-agent plan reconfiguration under local ltl specifications," *The International Journal of Robotics Research*, vol. 34, no. 2, pp. 218–235, 2015.
- [28] —, "Distributed plan reconfiguration via knowledge transfer in multi-agent systems under local ltl specifications," in *2014 IEEE International Conference on Robotics and Automation (ICRA)*, 2014, pp. 4304–4309.
- [29] D. Tian, H. Fang, Q. Yang, and Y. Wei, "Decentralized motion planning for multiagent collaboration under coupled ltl task specifications," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 52, no. 6, pp. 3602–3611, 2021.
- [30] Z. Chen, L. Li, and Z. Kan, "Distributed task allocation and planning under temporal logic and communication constraints," *IEEE Robotics and Automation Letters*, vol. 9, no. 7, pp. 6536–6543, 2024.
- [31] C. H. Fua and S. S. Ge, "Cobos: Cooperative backoff adaptive scheme for multirobot task allocation," *IEEE Transactions on Robotics*, vol. 21, no. 6, pp. 1168–1178, 2005.
- [32] S. Choudhury, J. K. Gupta, M. J. Kochenderfer, D. Sadigh, and J. Bohg, "Dynamic multi-robot task allocation under uncertainty and temporal constraints," *Autonomous Robots*, vol. 46, no. 1, pp. 231–247, 2022.
- [33] V. C. Kalempe, L. Piardi, M. Limeira, and A. S. de Oliveira, "Multi-robot preemptive task scheduling with fault recovery: A novel approach to automatic logistics of smart factories," *Sensors*, vol. 21, no. 19, p. 6536, 2021.
- [34] C. Belta, B. Yordanov, and E. A. Gol, *Formal methods for discrete-time dynamical systems*. Springer, 2017, vol. 89.
- [35] X. Luo, S. Xu, R. Liu, and C. Liu, "Decomposition-based hierarchical task allocation and planning for multi-robots under hierarchical temporal logic specifications," *IEEE Robotics and Automation Letters*, 2024.
- [36] Z. Liu, M. Guo, and Z. Li, "Time minimization and online synchronization for multi-agent systems under collaborative temporal logic tasks," *Automatica*, vol. 159, p. 111377, 2024.
- [37] C. Bettstetter, H. Hartenstein, and X. Pérez-Costa, "Stochastic properties of the random waypoint mobility model," *Wireless networks*, vol. 10, no. 5, pp. 555–567, 2004.
- [38] P. Gastin and D. Oddoux, "Fast ltl to büchi automata translation," in *Computer Aided Verification*, G. Berry, H. Comon, and A. Finkel, Eds. Berlin, Heidelberg: Springer Berlin Heidelberg, 2001, pp. 53–65.